diffmrc Manual

Reference, methods, and verification records

Version 0.1

The reference document for the diffmrc package: installation and configuration, the API surface, protocol details, the design of the differentiable chain and its second derivatives, the verification records, benchmark results, and the development architecture. The companion *Tutorial* covers first use.

Contents

I User reference **4**

1 Overview and package layout **5**
1.1 Design principles . . . 5
1.2 Package layout . . . 5
1.3 How the documents are organized . . . 6

2 Installation and configuration **7**
2.1 Supported environment . . . 7
2.2 Runtime configuration . . . 7
2.3 Verifying the installation . . . 7
2.4 Optional: running the figures . . . 8
2.5 Known environment friction . . . 8

3 API reference **9**
3.1 core: the numerical kernel . . . 9
3.2 analysis: NumPy only . . . 9
3.3 policy: selection rules . . . 10
3.4 driver: the differentiable pipelines . . . 10
3.5 cli . . . 10

4 Protocols and options **11**
4.1 Orbital-point convergence . . . 11
4.2 State averaging . . . 11
4.3 Active-space windows . . . 12
4.4 The e2_scale switch . . . 12
4.5 Differentiation settings . . . 12

II Methods and records **13**

5 The differentiable chain **14**
5.1 Parameterization . . . 14
5.2 The two factory families, and why . . . 14
5.3 Inside the chain . . . 14
5.4 What is verified, and against what . . . 15

6 Second derivatives **16**
6.1 The nuclear Hessian by implicit differentiation . . . 16
6.2 The second-order stages never materialize the Hessian . . . 16
6.3 Degenerate pairs and the derivative basis . . . 16
6.4 From the Hessian to frequencies, and one cautionary record . . . 17

7 Verification records **18**

8 Benchmarks **19**

9 Known limitations **20**

III Development **21**

10 Architecture and contracts **22**

11 Reproducing the paper **23**

Part I

User reference

Chapter 1

Overview and package layout

1.1 Design principles

Three commitments shape the package, and most of its structure follows from them.

The first is that derivatives are never written by hand. Every nuclear gradient and second derivative is obtained by differentiating the energy code path with JAX. This is what makes the CASSCF, state-averaged CASSCF, and NEVPT2 layers share one implementation of the physics: a correction to the energy expression propagates to all of its derivatives without a separate derivation.

The second is that the package is layered, and the layer boundaries are tested. The kernel cannot import the layers above it; the analysis layer cannot import the kernel at all. The contract is enforced in the test suite (see Chapter **??**), so a violation is a failing test rather than a matter of review discipline.

The third is that convergence is reported, not assumed. Optimizers return the gradient norm at the point they stopped at, and the package distinguishes "the optimizer stopped improving" from "the point is stationary." Callers are expected to carry that distinction into whatever they claim about a result.

1.2 Package layout

```
src/diffmrc/
  core/       numerical kernel: rotations, CASCI, CASSCF, NEVPT2,
              stationary points, the frozen source of the chain
  analysis/   post-processing that reads converged objects: entropies,
              mutual information, spin and state identification,
              orbital identity and frontier windows; NumPy only
  policy/     selection rules over analysis output: composition,
              entropy-based and ranked orbital selection, verification
  driver/     composition layer: assembles the kernel into the
              differentiable SCF -> CASSCF -> NEVPT2 pipelines,
              orbital-point convergence, frozen-geometry factories
  report/     result objects and rendering
  cli/        the command-line entry point
```

The dependency direction runs downward only: `core` at the bottom, `cli` at the top. A user script may import any layer; the layers themselves may not import upward. This is what allows the analysis layer to be tested and used with nothing but NumPy installed.

1.3 How the documents are organized

This manual is the reference. Part I covers installation, the API surface, and the protocol details that the code reports but does not enforce. Part II carries the methods: the design of the differentiable chain, the second-derivative construction, the verification records against independent implementations, the benchmark results, and the consolidated limitation list. Part III is for development: layer contracts, the test architecture, and the reproduction steps for the paper.

The companion *Tutorial* is the on-ramp: installation, a first CASSCF gradient, state-averaged workflows, active-space selection, and a full benchmark walkthrough, each with runnable code.

Chapter 2

Installation and configuration

2.1 Supported environment

The package is developed and verified on Linux and Windows with CPython 3.11, on CPU. The Tutorial (Chapter 2) gives the minimal recipe; this chapter records the constraints behind it.

Dependency	Constraint	Why it is pinned
Python	≥ 3.10	`int.bit_count` in the determinant tools
numpy	≥ 1.24	analysis layer, reference arithmetic
scipy	≥ 1.10	L-BFGS-B and trust-region stages
pyscfad	$\geq 0.3.3$	provides the traceable SCF and integral path
pyscf	≥ 2.5	reference energies and comparisons
jax	≥ 0.4	array layer; the exact pin is pyscfad's
matplotlib	any recent	figure sources only

`pyscfad` pins a specific JAX version, which is why the package should live in its own environment. Mixing it into an environment with a different JAX pin produces failures that look like numerical bugs and are not.

2.2 Runtime configuration

Three settings are load-bearing.

`jax_enable_x64 = True`. The package is validated in double precision. With the default single precision the errors are not loud: gradients lose several digits and some convergence gates become unreachable, while energies look plausible.

`JAX_PLATFORMS = cpu`. The CPU platform is the verified configuration. On a machine with a CUDA device, leaving the platform unset selects the GPU, where the solver path used by the orbital canonicalization raises an error unrelated to the chemistry. Set the variable before importing JAX; setting it afterwards has no effect.

Thread environment variables. The driver assumes it owns the thread budget it was given. Setting the five BLAS thread variables keeps multiple concurrent calculations from oversubscribing a node; their absence does not corrupt results, it makes timing meaningless and slows everything.

2.3 Verifying the installation

```
pytest tests/ -v
lint-imports        # or: pytest tests/test_layer_contracts.py
```

The suite is the primary artifact to look at: it contains the first-order and second-order derivative checks against finite differences, the cross-validation against PySCF, and the layer contracts. A passing suite on your machine is the intended proof that the environment is sound.

2.4 Optional: running the figures

The figure sources under `figures/` import matplotlib and read JSON results produced by the scripts; they require no additional configuration. They are not part of the package tests.

2.5 Known environment friction

- The first evaluation of a freshly built node of the chain triggers XLA compilation that can take minutes. This is expected; a stopwatch on a single evaluation will mislead.
- pyscfad's `Mole` objects cannot cross a `jax.jit` boundary. The driver layer offers frozen-geometry factories for jitted contexts; the factory docstrings state which is which.
- On shared nodes, leave headroom for the owner of the other processes. The memory profile of this package is dominated by compilation- and evaluation-time buffers of the differentiated chain, not by the integrals, and it grows steeply with the number of rotation parameters.

Chapter 3

API reference

This chapter lists the entry points a user is expected to reach for, layer by layer. It is a map, not a transcription: the authoritative signature of every function is its docstring in the source, and the module-level `__all__` lists are the canonical inventories. Entries marked *stable* have been exercised by the verification records in Chapter **??**; the rest follow the same conventions but have lighter coverage.

3.1 core: the numerical kernel

Entry point	Purpose
`rotation.block_rotation(t, nmo, pairs)`	rotation matrix from the parameter vector
`orblib.orbital_pairs(ncore, nact, nmo)`	the non-redundant rotation pairs
`casci.cs_casci(...)`	CASCI energies; state-averaged branch with `nroots`, `return`
`optimizer.optimize_theta_scipy(...)`	L-BFGS-B stage
`optimizer.optimize_theta_hand(...)`	the fallback when scipy raises
`grad.make_energy_fn / make_grad_fn`	energy and nuclear-gradient callables
`grad.solve_theta_star(...)`	the two-stage orbital optimization
`grad.nuclear_gradient(mol, grad_fn)`	the differentiated gradient
`stationary.refine_stationary_point(...)`	damped-Newton refinement (Newton–CG step)
`stationary.curvature_report(H)`	the saddle-point check
`stationary.StopReason / StationaryPoint`	convergence verdict types
`nevpt2.impl.cas_dets_for2(...)`	determinant bitmaps of the active space
`nevpt2.impl.canonicalize_jax(...)`	orbital canonicalization on the differentiated path
`nevpt2.impl.nevpt2_e2_full2(...)`	the frozen NEVPT2 contraction

The kernel is frozen: later layers build on it and corrections to it go through the verification records, not through ad-hoc edits.

3.2 analysis: NumPy only

Entry point	Purpose
`states.state_groups / select_groups`	roots to physical states; state selection
`states.spin_square / det_abs_sz`	exact $\langle S^2 \rangle$ and $\lvert M \rvert$ of determinants
`orbitals.frontier_report / classify_orbital`	frontier identities; window suggestion
`orbitals.heavy_atoms / p_axis_profile`	atom keys; per-axis p profiles
`ci.occupations_from_ci / entropy_from_ci`	occupations and entropies from a CI vector
`ci.pair_density / mutual_information`	two-orbital reductions and their correlations
`rdm.entropy_from_rdms`	the density-matrix route to the same quantities
`diagnostics.is_single_reference / zs1`	multireference diagnostics
`plateau.detect_plateaus`	entropic plateaus of an orbital set

This layer imports nothing from the kernel; it operates on the determinant-bitmap convention and on arrays, and its tests run with NumPy alone.

3.3 policy: selection rules

The policy layer turns analysis output into selections: composition-based rules, entropy-based ordering, the ranked orbital selection with its singly-occupied and virtual-removal options, and the verification helpers. The module names are the inventory (`policy.composition`, `policy.entropy`, `policy.ranking`, `policy.rules`, and their siblings); the docstrings state the conventions each rule follows.

3.4 driver: the differentiable pipelines

Entry point	Purpose
`nevpt2.make_nevpt2_joint_fn(...)`	(mol, θ) SCF→CASSCF→NEVPT2, geometry-diffe
`nevpt2.make_nevpt2_theta_fn(...)`	frozen geometry, θ only; jit-safe
`nevpt2.make_nevpt2_sa_fn(...)`	state-averaged joint factory
`nevpt2.make_nevpt2_sa_theta_fn(...)`	state-averaged, frozen geometry, jit-safe
`nevpt2.sa_cas_solution(...)`	state-averaged CAS solution and CI vectors
`nevpt2.sa_nevpt2_state_table(out, dets)`	grouped physical-state table
`nevpt2.sa_vertical_excitations(states)`	per-state vertical excitation energies
`orbitals.converge_orbital_point(E_theta, npar)`	two-stage convergence with refinement and polish
`orbitals.hessp_of(E_theta)`	Hessian-vector product for the trust-region stage

The two factory families exist because of a pyscfad constraint: the `Mole`-based factories cannot be traced inside `jax.jit`, and the theta-only factories are built for exactly that context. When in doubt about which to call, ask whether the geometry is fixed; if it is, use the theta-only factory.

3.5 cli

`python -m diffmrc.cli` runs an end-to-end calculation from a geometry file; `--help` lists the options. The command-line entry is the least settled surface of the package.

Chapter 4

Protocols and options

The package reports convergence and protocol evidence; it does not enforce it. This chapter collects the settings that decide whether a number is trustworthy, with the values the benchmark records were produced under. Where a default can be changed by a caller, the change belongs next to the number it produced, because several of the gates below are reachable at more than one tolerance and the results differ at the margins.

4.1 Orbital-point convergence

The two-stage protocol separates two different failure modes: the optimizer that can no longer lower the energy, and a point that is not stationary. The stages and their defaults:

Setting	Value	Meaning
`optimize_tol`	10^{-6}	L-BFGS-B target; loose on purpose
`refine_target`	10^{-11}	the Newton target, and the verdict
`max_steps`	2000	L-BFGS-B iteration cap
`refine_max_iter`	80	damped-Newton iteration cap
`_CG_tol` / `_CG_maxiter`	10^{-6} / 40	Newton–CG budget for the step solve

The verdict is the gradient norm at the point the chain stopped at, classified by the stop reason:

`StopReason`	Reading
`CONVERGED`	gradient below the refine target
`STALLED_GRADIENT`	a step no longer reduced the gradient
`GUARD_STOPPED`	no damped step lowered the energy
`ITERATION_CAP`	budget exhausted; re-check before claiming convergence

Two details of the Newton stage are load-bearing and easy to lose when reimplementing. The Hessian enters only through Hessian-vector products — the explicit `jax.hessian` broadcasts every intermediate over n_{par}^2 tangent pairs and was killed by a 402 GB allocation at $n_{\mathrm{par}} \approx 500$ — and the downhill acceptance test is not optional: an unguarded refinement moved an energy from -76.07 to $+1.04$ Ha in this project's history and printed no error.

4.2 State averaging

The state-averaged factories take `nroots` and `state_weights`; the weights are normalized and must have one entry per root. The measured effect of between-group equal weights (the second pass) on the reported excitation energies was at or below 0.1 eV for the benchmark systems,

while the second pass usually tightens the final orbital gradient by orders of magnitude. The benchmark records quote which pass each number came from.

Two root-table facts belong with every state-averaged number. The eigensolver returns roots, so a triplet travels as three degenerate roots; and a per-component NEVPT2 correction can be undefined (`nan`) for a pure $|M| = 1$ component, in which case the group's representative is the lowest component with a finite value. `sa_nevpt2_state_table` performs both bookkeeping and returns the evidence.

4.3 Active-space windows

The frontier-window rules are stated in the Tutorial (its chapter on choosing an active space). The part that belongs to a protocol table is the bookkeeping: the electron count follows the window,

$$n_{\mathrm{ele}} = N_{\mathrm{ele}} - 2\, n_{\mathrm{core}}, \tag{4.1}$$

and a window that widens to three orbitals for the linear two-pi and amide patterns therefore carries four active electrons, not two. A hard-coded two here produces a wrong ground state that the energy alone will not flag.

4.4 The e2_scale switch

Both NEVPT2 factories accept `e2_scale`. With `e2_scale = 0` the perturbation correction is switched off and the call reduces to the CASSCF energy of the same chain — the identity used by the test suite to pin the joint and state-averaged factories against the validated single-root path. With the default value of 1.0 the correction is included. There is no other intermediate value with a physical meaning; the switch exists for verification.

4.5 Differentiation settings

The theta-only factories are safe inside `jax.jit`; the `Mole`-based ones are not, and the failure mode is a tracing error at compile time rather than a wrong number. The second-order paths (the refinement, the trust-region polish) differentiate the gradient with forward-mode `jvp` calls; a caller wrapping the whole convergence in `jit` should expect the compile of those nodes to dominate the first evaluation.

Part II

Methods and records

Chapter 5

The differentiable chain

This chapter describes how the package makes a multireference energy differentiable, and records the numerical facts that shaped the design. It is the software-side companion to the methods paper; the derivations live there, the implementation constraints live here.

5.1 Parameterization

Orbital variation enters through a rotation parameter vector θ: the occupied–virtual mixing is generated by the non-redundant pair list of `orblib.orbital_pairs`, and `rotation.block_rotation` turns θ into the rotation matrix applied to the reference orbitals. Three pair families are included: core–active, core–virtual, and active–virtual, for every orbital outside the active set.

The active–active rotations are omitted deliberately: they are redundant against the CI step, which is complete inside the active space. The core–virtual family is *not* omittable. A parameterization that drops it stops at a point that is not stationary; the defect was measured at the level of a few hundredths of a kcal/mol on a test system — small, systematic, and invisible without an independent reference. The pair list is the contract; do not prune it for speed.

5.2 The two factory families, and why

Every differentiable entry point comes in two forms.

The *joint* factories (`make_nevpt2_joint_fn` and its state-averaged sibling) take the molecule and θ and run the reference SCF inside the call. They are the geometry-differentiable path: the nuclear coordinates enter through the integrals, and JAX differentiates the whole composition.

The *theta-only* factories (`make_nevpt2_theta_fn`, `make_nevpt2_sa_theta_fn`) compute the reference SCF and the integrals once, at construction, and return a callable of θ alone. They exist for one hard reason: this build of pyscfad cannot trace its SCF inside `jax.jit`, and the orbital optimizer wraps its objective in `jit`. A joint factory called from inside the optimizer raises a tracing error; the theta-only factory is the supported path there. The two families funnel through one shared component builder, so they cannot drift apart numerically — the test suite pins them against each other at fixed geometry.

5.3 Inside the chain

A single evaluation composes four numerically delicate pieces.

The MO integral transform runs in four contraction steps. The four-operand form of the two-electron transform is numerically weaker on XLA at large sizes, and the discrepancy is not cosmetic: on an 96^4 integral array the one-shot form showed absolute errors around 10^{-6}, which the near-degenerate gaps of a linear molecule's generalized Fock matrix amplify into a completely different eigenvector basis inside the degenerate subspace. The energy is insensitive

to that; the theta gradient differed by eight percent between the two contraction orders. The stepwise form is therefore the implementation, not an optimization.

Canonicalization is differentiable and state-aware. The NEVPT2 denominators are defined on the canonical (natural-orbital, sub-Fock-diagonalized) set, so the contraction runs on canonicalized orbitals produced inside the differentiated path. The state-averaged variant rotates with the average density of the state-averaged solution; at one root it reproduces the ground-state canonicalization bit for bit, which is how the two are pinned against each other.

Per-state energies are read out of the eigensolver's spectrum. The state-averaged solver returns the mean of the lowest roots; the per-state energies are recovered from the running means by the sliding-mean inversion $E_k = k\,M_k - (k-1)\,M_{k-1}$. The number of roots is silently clamped by the size of the determinant space, so the inversion is performed over the effective root count, not the requested one.

Degeneracies are handled at the decomposition level. An eigensolve with degenerate eigenvalues has no unique derivative basis; the kernels use degeneracy-safe decompositions throughout, and the choice of cluster tolerance is a protocol decision with its own limitation entry (Chapter **??**).

5.4 What is verified, and against what

The chain's verification record is deliberately built on invariance and cross-implementation checks rather than on agreement with any single reference: the CASSCF gradient is validated against invariance conditions and against an independent derivative-free route; the NEVPT2 energies against an independent implementation; the state-averaged addition against the single-root path at one root. Chapter **??** lists the records with their measured deviations.

Chapter 6

Second derivatives

The package differentiates its energy chain twice, so that harmonic frequencies and curvature checks come out of the same code path that produced the gradients. This chapter records the construction and the three numerical facts that forced their design.

6.1 The nuclear Hessian by implicit differentiation

Differentiating through a converged orbital optimization naively — differentiating the unrolled iteration — scales with the number of iterations and inherits every numerical accident of the solver. The production route instead treats the optimized orbital parameters as an implicit function of the geometry, $\theta^*(R)$, and assembles the second derivative from the partial derivatives of the energy at the converged point plus a response term built from the orbital Hessian.

Two implementation facts came out of the verification work. The response solve must use a pseudoinverse rather than a plain solve: the orbital Hessian at a converged point is singular in the directions the optimizer is free to ignore, and the plain solve either fails or returns noise. And the implicitly assembled Hessian must be pinned against the alternative construction: the two routes agree to 1.6×10^{-11} on the validation systems, with the implicit route costing about a third of the unrolled one.

6.2 The second-order stages never materialize the Hessian

The orbital refinement (damped Newton) and the trust-region polish need the Hessian only through its action on vectors. The earlier implementation built it explicitly with `jax.hessian`, which is forward-over-forward: every intermediate of the chain is broadcast over the square of the parameter count. At five hundred parameters that construction requested a single 402 GB allocation and died. Both stages now receive Hessian-vector products — a forward-mode derivative of the gradient — which carry one tangent per reverse pass: the refinement solves its Newton step by conjugate gradients on those products, and the trust-region stage consumes them through the optimizer's `hessp` interface. The budgets are deliberately shallow (tolerance 10^{-6}, forty iterations): an inexact Newton step feeds a damping ladder and a downhill acceptance test that correct it, and the deep solver cost hours of product evaluations per molecule for no change in the converged point.

6.3 Degenerate pairs and the derivative basis

A symmetry-degenerate pair of orbitals has no canonical derivative basis: the eigenvectors inside the degenerate subspace are arbitrary, so a naive derivative of a decomposition returns a quantity that depends on the solver's internal ordering. For a linear molecule this is not an edge case — the π pairs are exactly degenerate, and a gradient assembled through an unguarded decomposition

returns NaN. The kernels use degeneracy-safe eigensolvers throughout the differentiated path, and the second-order routes avoid differentiating decompositions where the response construction allows it.

6.4 From the Hessian to frequencies, and one cautionary record

Harmonic frequencies follow from the mass-weighted Hessian by the standard normal-mode analysis, using PySCF's driver on a Hessian this package assembles. The conversion has a layout trap that cost this project a wrong number: the Hessian must be handed over in the (atom, coordinate) block layout the thermodynamics module expects, not as a plain reshape. The wrong layout comes out plausible and silent — for CO at 6-31G it produced 1501.8 cm^{-1} against the correct 2293.5 cm^{-1}.

The frequency route is anchored against an independent value: the CO molecule at 6-31G gives 2294.2 and 2293.5 cm^{-1} from the two assembly routes, against an external anchor at 2293.5 cm^{-1}. When a validation target sits near zero, do not reach for finite differences: this project measured finite-difference errors of over one hundredfold on a near-zero mixed derivative, growing as the step shrank. Use an invariance or an analytic cross-check instead.

Chapter 7

Verification records

Chapter 8

Benchmarks

Chapter 9

Known limitations

Part III

Development

Chapter 10

Architecture and contracts

Chapter 11

Reproducing the paper